

SMART: LLM-Augmented Hybrid Retrieval for Dynamic Product Ads

Congfei Zhang
Snap Inc.
Bellevue, WA, USA
czhang3@snapchat.com

Jingxiao Ma
Snap Inc.
Bellevue, WA, USA
jma3@snapchat.com

Xiaodong Liu
Snap Inc.
Bellevue, WA, USA
xliu9@snapchat.com

Hsiang-wei Chao
Snap Inc.
Seattle, WA, USA
hchao@snapchat.com

Siman Wang
Snap Inc.
Bellevue, WA, USA
swang7@snapchat.com

Ge Liu
Snap Inc.
Bellevue, WA, USA
gliu@snapchat.com

Shantanu Aggarwal
Snap Inc.
Palo Alto, CA, USA
saggargwal@snapchat.com

Vincent Zhang
Snap Inc.
Bellevue, WA, USA
wzhang7@snapchat.com

Meghana Missula
Snap Inc.
Palo Alto, CA, USA
mmissula@snapchat.com

Rachel Liao
Snap Inc.
New York, NY, USA
rliao2@snapchat.com

Zichu Li
Snap Inc.
Palo Alto, CA, USA
zli12@snapchat.com

Xiao Bai
Snap Inc.
Palo Alto, CA, USA
xbai@snapchat.com

Yunzhi Zhou
Snap Inc.
Palo Alto, CA, USA
yzhou10@snapchat.com

Yajun Wang
Snap Inc.
Palo Alto, CA, USA
ywang30@snapchat.com

Zhe Liu
Snap Inc.
Palo Alto, CA, USA
zliu11@snapchat.com

Jinchao Li
Snap Inc.
Bellevue, WA, USA
jli18@snapchat.com

Yu Zhang
Snap Inc.
Palo Alto, CA, USA
yzhang3@snapchat.com

Abstract

Dynamic Product Ads (DPA) require retrieving relevant items from multi-million product catalogs, balancing two competing objectives: retargeting (re-surfacing known interests) and prospecting (discovering new categories). While Large Language Models (LLMs) capture semantic intent better than traditional embedding models, deploying them at scale introduces prohibitive inference costs and lexical mismatch issues. Through controlled experiments on millions of users, we demonstrate a critical retrieval decomposition: rule-generated queries excel at retargeting on a lexical BM25 index, while LLM-generated queries excel at prospecting on a dense ANN index. Building on this, we propose SMART (SeMantic-aware Adaptive ReTrieval). To manage costs, a lightweight quality gate identifies coverage gaps in initial keyword results, adaptively routing only the ~10% of users who benefit from semantic prospecting to the LLM path. Offline evaluation demonstrates that this gated approach captures the bulk of semantic prospecting gains in Relevance Score while maintaining competitive re-targeting performance at a 90% reduction in LLM costs. Finally, in a 2-week online A/B test at Snap, SMART improved the ad conversion rate by +27.6% over a strong embedding-based baseline.

CCS Concepts

• **Information systems** → **Language models; Recommender systems; Online advertising.**

Keywords

Dynamic Product Ads, Large Language Models, hybrid retrieval, query generation, adaptive routing, cost-aware serving

ACM Reference Format:

Congfei Zhang, Jingxiao Ma, Xiaodong Liu, Hsiang-wei Chao, Siman Wang, Ge Liu, Shantanu Aggarwal, Vincent Zhang, Meghana Missula, Rachel Liao, Zichu Li, Xiao Bai, Yunzhi Zhou, Yajun Wang, Zhe Liu, Jinchao Li, and Yu Zhang. 2026. SMART: LLM-Augmented Hybrid Retrieval for Dynamic Product Ads. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Dynamic Product Ads (DPA) drive personalized recommendations by analyzing a user’s shopping behavior. The core retrieval challenge involves sifting through millions of items to identify the most relevant products based on a user’s browsing, cart, and purchase history. This task is uniquely difficult because it requires balancing two competing objectives. On one hand, systems must excel at retargeting—re-surfacing products from known brands and categories to drive immediate conversions. On the other hand, they must support prospecting—discovering relevant items in new categories to expand the user’s consideration set.

Existing DPA retrieval systems rely heavily on collaborative filtering or embedding-based models [2, 11]. While these approaches capture behavioral similarity, they struggle to express semantic intent (e.g., inferring “running accessories” from an athletic-footwear

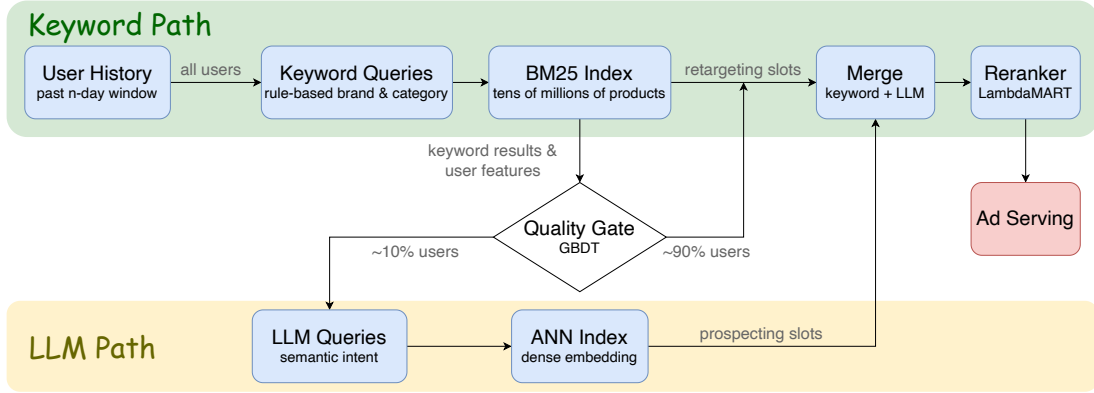


Figure 1: SMART system architecture. All users receive rule-generated queries for BM25 retargeting. The quality gate evaluates coverage gaps and routes ~10% of users to LLM prospecting queries on a dense ANN index. Candidates are merged, reranked, and sent to ad serving.

purchase). Large Language Models (LLMs) offer a promising alternative, as their pre-trained world knowledge allows them to interpret user histories and generate semantically rich search queries [27, 29]. However, deploying LLMs in ad retrieval introduces three major challenges. First, there is a query-backend mismatch: LLM-generated text often fails to align with the strict lexical patterns required by traditional BM25 indexes [15, 18]. Second, invoking an LLM for every user is cost-prohibitive and impractical under strict millisecond latency constraints [17]. Finally, the industry lacks a controlled, production-scale comparison detailing when LLM-generated queries outperform simpler rule-based approaches.

To address these challenges, we propose SMART (SeMantic-aware Adaptive ReTrieval), which serves every user via a zero-cost rule-generated keyword pass and uses a lightweight quality gate to audit candidate coverage—invoking an LLM-driven dense retrieval path only for users left under-covered, as Figure 1 illustrates. We show that rule-generated and LLM-generated queries are not substitutes, but serve distinct, complementary purposes. Rule-generated queries emit exact tokens from the user’s history, making them ideal for retargeting on a lexical BM25 index. Conversely, LLMs are uniquely suited for semantic prospecting on a dense Approximate Nearest Neighbor (ANN) index, where their world knowledge surfaces complementary products that cannot be extracted from the user’s history. This gated pairing achieves the best retrieval quality while reducing LLM costs by 90%.

We dissect the efficacy of SMART through extensive offline experiments and a 2-week online A/B test against Snap’s existing embedding-based baseline—a two-tower user-to-product (U2I) model with ANN retrieval. Our main contributions are:

- **A retargeting–prospecting decomposition:** We validate on a large-scale production cohort that rule-generated queries excel at retargeting (measured by CatRecall@K), while LLMs excel at prospecting (measured by Relevance Score).
- **A cross-metric evaluation:** We empirically demonstrate that CatRecall@K and Relevance Score yield opposite rankings on the same systems, confirming that retargeting and prospecting necessitate different query strategies and backends.

- **A learned output-auditing quality gate:** We introduce a light-weight GBDT router that audits the output of the zero-cost keyword pass by measuring coverage gaps relative to user historical interests. By adaptively routing only the ~10% of users with the highest predicted coverage deficits to the LLM path, SMART captures peak retrieval quality while achieving a 90% reduction in LLM inference costs.
- **Production impact:** We detail the deployed SMART system, which successfully routes rule-generated queries to BM25 and LLM queries to ANN, driving a +27.6% relative improvement in conversion rate during a 2-week online A/B test at Snap.

2 Related Work

LLM-Based Query Expansion. Early generative query expansion techniques like Query2Doc [27] append LLM-generated pseudo-documents to user queries, demonstrating consistent improvements on dense retrieval benchmarks but showing less reliable gains on sparse indices. Similarly, architectures like HyDE [15] generate hypothetical documents specifically for dense embedding vector spaces while avoiding lexical matching entirely due to term-frequency discrepancies. Comparative evaluations across sparse and dense retrievers confirm that LLM query expansions provide significantly larger and more stable performance lifts when paired with semantic backends [18]. Rather than direct query execution, frameworks like InPars [5] restrict the use of generative text to synthesizing offline training data for neural models. Collectively, these methodologies indicate that synthetic, LLM-generated text aligns far better with latent semantic spaces than with strict term-matching mechanics. Our work provides empirical confirmation of this behavior at production scale within a commercial catalog framework.

LLM Query Expansion Failures. Generative query expansion is susceptible to specific systemic vulnerabilities, particularly when handling unfamiliar topics or ambiguous source inputs that trigger narrow, biased refinements [1]. Furthermore, rewriting queries without factoring in the specific index capabilities of the underlying

retrieval engine can produce fundamentally invalid retrieval plans. To mitigate this issue, probe-based architectures have been proposed to inspect the target index state before executing a plan [9]. Our observation that LLM-generated retargeting queries function as degraded paraphrases of exact keywords serves as a clear instance of this backend-agnostic failure mode when applied to highly structured product catalog fields.

Cost-Aware LLM Routing. Managing the high computational overhead of generative models is typically addressed via dynamic routing cascades like FrugalGPT [8], which uses learned scorers to direct inputs to cheaper or more complex model tiers to achieve up to a 98% reduction in cost. Similarly, frameworks like RouteLLM [21] utilize human preference data to train efficient multi-tier routers, while systems like AutoMix [3] apply self-verification loops to decide when an output requires escalation to a larger foundational model. Operational deployments like Hybrid LLM [12] establish thresholds to split inference workloads between edge devices and cloud infrastructure, and architectures like COEF-VQ [13] leverage cheap entropy-based filtering to gate heavy multimodal pipelines. Alternatively, Self-RAG [4] introduces internal critique tokens to dictate exactly when external knowledge retrieval is warranted. While most of these frameworks route between language models of varying sizes based on properties of the input, SMART applies the routing idea at a different decision point: a learned gate audits the *output* of a zero-cost deterministic keyword pass and decides whether invoking a generative engine is warranted at all.

Product Retrieval for Ads. DPA systems typically adapt the collaborative filtering and embedding-based candidate-generation paradigms developed for large-scale recommenders [11]. Hybrid sparse-dense retrieval is likewise established practice, from unsupervised rank fusion [10] to systematic analyses of sparse, dense, and hybrid representations [20]; we treat the hybrid itself as prior art and focus on when each path helps and whether the expensive path should run at all. Abutbul et al. [2] describe audience prospecting for DPA at Yahoo using retargeting and search-based approaches without LLM query generation. Hu et al. [17] note that deploying LLMs directly in online ad systems is “impractical due to millisecond latency constraints,” using LLMs only for offline tasks. Pinterest [26] integrates LLMs for search relevance scoring but not for query generation. To our knowledge, no prior work provides a controlled study of when LLM-generated queries help or hurt in production ad retrieval.

3 Preliminary Study: Retargeting vs. Prospecting

Before describing the architecture of SMART, we present a diagnostic production study run on live traffic logs. This empirical analysis exposes the core behavior and limitations of traditional lexical retrieval and generative retrieval paths, defining the requirements for an optimized production setup.

3.1 Diagnostic Setup and Metrics

We sample an offline evaluation group of over 20K unique users from production DPA logs across multiple countries. For each user, their full interaction behavior (browsing, cart, and purchase events)

Table 1: Lexical retrieval comparison on BM25. Rule-generated keyword queries achieve higher CatRecall@K at every evaluation depth, reflecting their structural alignment with token-frequency matching.

K	Keyword (baseline)	LLM retarget+prospect	Δ rel.
5	41.0%	34.5%	−15.9%
20	44.4%	37.1%	−16.4%
200	56.0%	42.5%	−24.1%

recorded up to day $T - 1$ serves as the sole input to query generation. This mirrors the deployed system, where no current-session context is used: query generation consumes the accumulated history alone, while the routing gate conditions on that history, static profile attributes, and the keyword-pass output (Section 4). The candidate configurations retrieve the top-200 products from an active catalog of tens of millions of listings, which are then evaluated against the user’s actual purchases documented on day T and against an offline LLM judge that emits a Relevance Score. All conditions share identical underlying indices, reranker models, and pipeline layouts to isolate query generation performance. Full implementation parameters are detailed in Section 4.3. Performance is tracked through two distinct, structurally opposing lenses:

- **CatRecall@K** (Retargeting Metric): The fraction of Google Product Categories (GPC) purchased on day T that are successfully surfaced within the top- K retrieved items. Category-level evaluation is matching at the natural granularity for DPA systems because exact product-ID recall is vanishingly sparse at scale.
- **Relevance Score** (Prospecting Metric): An LLM-as-a-judge [28] evaluation using Gemini 3 Flash [16] (with thinking mode enabled) scores the top-10 retrieved items against the user’s full shopping history on a 0–3 scale (3 = highly relevant, 2 = matches intent, 1 = adjacent, 0 = irrelevant). This metric explicitly captures semantic alignment to broader intent, rewarding discovery within categories the user has not yet purchased.

We evaluate two distinct query strategies: *rule-generated keywords* and *LLM-generated queries* across two retrieval backends: a Tantivy-based BM25 lexical index and a dense FAISS-based Approximate Nearest Neighbor (ANN) index.

3.2 Token-Frequency Limits on Lexical Search

We first cross-examine the behavior of both query strategies on a shared BM25 index to isolate the characteristics of pure lexical matching.

Table 1 reveals an explicit performance gap: the deterministic Keyword baseline outperforms the generative LLM strategy across every depth, securing a +15.9% relative advantage at $K = 5$ and a +24.1% lead at $K = 200$. This behavior is a direct consequence of index mechanics: BM25 functions as a token-frequency matcher that weights fields by inverse document frequency. Rule-generated queries emit exact brand and category strings drawn directly from user text logs, matching indexed product metadata exactly. When an LLM paraphrases this precise intent (e.g., expanding a brand title into natural description strings), it introduces vocabulary permutations that do not exist verbatim within catalog fields, effectively

Table 2: Cross-metric backend comparison. CatRecall is evaluated at $K = 200$; Relevance Score is checked at $K = 10$ on a 0–3 scale. Rule-generated keyword + BM25 leads on CatRecall (retargeting), while LLM + ANN dominates on Relevance Score (prospecting).

System	CatRecall@200 (Δ rel.)	Rel. Score (Δ rel.)
Keyword + BM25 (base)	56.0%	1.95
Keyword + ANN	53.9% (−3.8%)	2.25 (+15.3%)
LLM + BM25	42.5% (−24.1%)	2.32 (+18.9%)
LLM + ANN	50.7% (−9.5%)	2.39 (+22.6%)

Table 3: Per-user impact of adding LLM queries to retrieval. Only 14.4% of users see clear metric improvement, while 73% achieve optimal coverage via exact keywords alone.

Outcome	% of users
Both CatRecall@200 and Relevance Score	2.6%
CatRecall@200 only	2.2%
Relevance Score only	9.6%
Either metric (helped)	14.4%
No change	73.0%
Declined on either metric	12.6%

diluting term frequency and degrading lexical precision. Thus, rule-generated keywords are superior for exact retargeting objectives.

3.3 Cross-Metric Index Inversion

The narrative completely reverses when moving candidates into a dense vector space. Table 2 maps both query generation styles across both the BM25 and dense ANN indices evaluated across both key metrics.

We observe a clear cross-metric inversion: LLM queries improve Relevance Score across any backend (+18.9% on BM25, +22.6% on ANN), yet the keyword rules cleanly dominate on CatRecall@200. Crucially, each query style is best served by its matched backend: switching from BM25 to ANN boosts CatRecall@200 by +19.3% relative for the LLM path, while the same switch costs keyword queries 3.8%. Because dense indices reward conceptual vector proximity over strict token matching, they enable the LLM queries to exploit pre-trained world knowledge, successfully surfacing highly relevant, out-of-history discovery items (prospecting). These findings confirm that keywords + BM25 and LLM + ANN serve completely different retrieval objectives.

3.4 Quantifying Gating Opportunities

While running both paths simultaneously for all incoming traffic would technically maximize candidate pools, two production constraints make a universal configuration impossible. First, global LLM inference introduces massive computational costs and violates strict millisecond ad serving response budgets. Second, our user-level impact analysis (Table 3) indicates that the benefits of semantic prospecting are heavily concentrated.

A striking 73.0% of users experience absolutely no metric change when generative expansion is appended, as their intent is already

Algorithm 1 Backend-Aware Keyword Query Generation

Require: User history $H_u = \{(e_i, t_i, p_i)\}$, budget n queries

Ensure: Query set Q_{kw}

```

1: for each brand  $b$  in  $H_u$  do
2:    $s(b) \leftarrow \sum_{e_i: \text{brand}(p_i)=b} w(e_i)$   $\triangleright$  Purchase=3, Add-to-cart=2, View=1
3: end for
4:  $B \leftarrow$  top- $(n/2)$  brands by  $s(b)$  descending
5: for each  $b \in B$  do
6:    $c_b \leftarrow$  most frequent leaf category for  $b$  in  $H_u$ 
7:    $Q_{kw} \leftarrow Q_{kw} \cup \{b \oplus c_b\}$   $\triangleright$  e.g., “Brand-X Running Shoes”
8: end for
9:  $C \leftarrow$  top- $(n - |Q_{kw}|)$  categories by frequency, excluding covered

10: for each  $c \in C$  do
11:    $Q_{kw} \leftarrow Q_{kw} \cup \{c\}$   $\triangleright$  e.g., “Baby Clothing”
12: end for
13: return  $Q_{kw}$ 

```

completely satisfied by the cheap keyword path. Processing universal traffic through an LLM would yield massive infrastructure waste and introduce candidate noise for the majority of requests. Nor can the downstream reranker fully absorb this noise: it demotes weakly matching items within the merged pool, but candidates displaced from the finite retrieval pool are unrecoverable—indeed, indiscriminate LLM augmentation lowers CatRecall@200 even after reranking (Table 6). This structural asymmetry motivates the proposed architecture: a framework that routes all traffic through the zero-cost keyword path, paired with an adaptive gate that conditionally invokes the expensive LLM vector path only for the targeted $\sim 10\%$ of users facing a candidate coverage gap.

4 The SMART System Architecture

The SMART system orchestrates a dual candidate generation loop managed by an online output-auditing routing layer, matching each query strategy to its empirically optimized index backend, as Figure 1 shows.

4.1 Multi-Stage Retrieval Pipeline

The core pipeline processes incoming requests across four sequential operational stages:

1. Query Generation. For a user history $H_u = \{(e_i, t_i, p_i)\}$, SMART combines a deterministic keyword query generator with a selective generative path. Algorithm 1 describes the rule-generated keyword component, which is explicitly constructed to be backend-aware, emitting tokens guaranteed to appear within the BM25 catalog fields. It tracks brand engagement by computing a score $s(b) = \sum w(e_i)$, where weights prioritize down-funnel conversions ($w(\text{Purchase}) = 3$, $w(\text{Add-to-Cart}) = 2$, $w(\text{View}) = 1$). The top $n/2$ brands by descending engagement are paired with their most frequent historical leaf categories to form exact search tokens, and remaining slots are filled with high-frequency category strings to broaden baseline coverage.

Algorithm 2 Adaptive Query Routing**Require:** User history H_u , gate model g , threshold θ **Ensure:** Retrieved product set $\mathcal{R}(u)$

```

1:  $Q_{kw} \leftarrow \text{KEYWORDQUERIES}(H_u)$   $\triangleright$  Algorithm 1, zero cost
2:  $\mathcal{R}_{kw} \leftarrow \text{BM25SEARCH}(Q_{kw})$ 
3:  $\mathbf{x} \leftarrow f(H_u, \mathcal{R}_{kw})$   $\triangleright$  Coverage gap features
4: if  $g(\mathbf{x}) \geq \theta$  then
5:    $Q_{llm} \leftarrow \text{LLM}(H_u, Q_{kw})$   $\triangleright$  coverage gap: prospecting queries
6:    $\mathcal{R}_{llm} \leftarrow \text{ANNSEARCH}(Q_{llm})$ 
7:    $\mathcal{R}(u) \leftarrow \text{RERANK}(\mathcal{R}_{kw} \cup \mathcal{R}_{llm})$ 
8: else
9:    $\mathcal{R}(u) \leftarrow \text{RERANK}(\mathcal{R}_{kw})$ 
10: end if
11: return  $\mathcal{R}(u)$ 

```

2. *Retrieval.* Keyword queries run against a Tantivy [24] BM25 [23] index built from the catalog. Each product document is indexed with title, brand, Google Product Category, product type, and a 5-level hierarchical semantic ID following the TIGER framework [22]. The semantic IDs are generated by fine-tuning a SigLIP2 [25] multimodal vision-language encoder on the Snap product catalog to produce product embeddings, then applying Residual Quantization k -means (RQ-KMeans) to discretize the continuous embedding space into a 5-level hierarchy—L1 captures broad product domains (e.g., “Electronics”), while L5 identifies near-duplicate products. When selectively invoked by the quality gate, semantic prospecting queries are processed via a single LLM call and executed against a dense ANN index managed via FAISS [14].

3. *Reranking.* Candidates retrieved from both paths are merged and scored using a LambdaMART [6] learning-to-rank model trained with LightGBM [19], followed by a Maximal Marginal Relevance (MMR) [7] layer to enforce brand, category, and semantic diversity.

4. *Ad Serving.* Top-ranked products are mapped to active advertiser product sets to ensure absolute alignment with live ad campaigns before serving.

4.2 Adaptive Quality Gate Mechanism

To bypass expensive generative processing for users already well-served by rule generated keywords, SMART places an adaptive classifier directly after the zero-cost keyword retrieval pass, deciding whether to invoke the generative prospecting path, as Algorithm 2 describes.

Problem Formulation. Let $\mathcal{R}_{kw}(u)$ denote candidates retrieved via the zero-cost keyword pass, and $\mathcal{R}_{smart}(u)$ denote the combined multi-backend candidate pool. The incremental retrieval benefit $\Delta(u)$ is defined as:

$$\Delta(u) = m(\mathcal{R}_{smart}(u)) - m(\mathcal{R}_{kw}(u)) \quad (1)$$

where $m(\cdot)$ represents retrieval quality via category recall. The routing objective maximizes global retrieval quality over all users subject to a strict maximum allowable LLM budget fraction B :

$$\max_{\theta} \sum_{u \in \mathcal{U}} m(\mathcal{R}_{\theta}(u)) \quad \text{s.t.} \quad \frac{|\{u : g(u) \geq \theta\}|}{|\mathcal{U}|} \leq B \quad (2)$$

Table 4: Quality gate feature groups. Bold groups are derived directly from the real-time keyword search results and serve as the most discriminative signals. Semantic L1/L2 cluster IDs are produced by hierarchically clustering SigLIP2 vision-language embeddings of catalog products.

Group	Key Features	Count
User history	Brand/category entropy, diversity ratio	15
Temporal	History span, events/day, recency	5
Purchase value	Avg/max purchase value	3
Match quality	Avg/min BM25 match scores	2
Category depth	GPC depth, L1 category count	3
Semantic history	Semantic L1/L2 cluster counts	3
Brand-category	Categories per brand, brands per cat	2
Query features	Query count, avg tokens, brand queries	3
Search results	Products, categories, coverage	7
Coverage gaps	Category/semantic gap, diversity	9
Profile	Age group, gender	2

where $\mathcal{R}_{\theta}(u) = \mathcal{R}_{smart}(u)$ if the gate score $g(u) \geq \theta$, and $\mathcal{R}_{kw}(u)$ otherwise.

Gate Architecture and Feature Engineering. The gate functions as a lightweight binary GBDT classifier [19] evaluating 54 real-time features across 11 structural groups (summarized in Table 4). Crucially, rather than relying solely on static user histories, the gate conditions its routing choice directly on the *output insufficiency* of the cheap keyword pass by measuring the structural coverage gap:

$$\text{gap}(u) = 1 - \frac{|\text{cat}(\mathcal{R}_{kw}(u)) \cap \text{cat}(H_u)|}{|\text{cat}(H_u)|} \quad (3)$$

where $\text{cat}(\cdot)$ maps retrieved items to their unique category taxonomies. A high gap indicates that a user’s diverse historical affinities have been displaced or unfulfilled within the top lexical matches, marking a requirement for generative prospecting. This output-auditing design distinguishes the gate from input-based cost-aware routers [8, 21], which decide from the input alone before any retrieval is attempted: here the router observes what the cheap pass actually retrieved before spending on the expensive path. Training labels are mapped as positive via an OR-of-ORs construction if SMART beats keywords on CatRecall@200 across either of two independent cycles, capturing stable lift potential while minimizing LLM stochasticity.

4.3 Core Implementation Specifications

The candidate generation infrastructure runs against a Tantivy BM25 index built from the active catalog. The generative path utilizes Gemini 2.5 Flash Lite via a single structured API call to output exactly three prospecting queries; retargeting is handled entirely by the rule-generated keyword path. The lightweight Flash Lite model is chosen for the cost-sensitive production generation path, whereas the heavier Gemini 3 Flash (thinking mode) serves only as the offline evaluation judge (Section 3.1). Conceptual representations are encoded using the 768-dimension gemini-embedding-001 model over a concatenated text string of product title, brand, description, and category attributes. Dense retrieval is executed over vector fields mapped via FAISS GPU IVFFlat.

Table 5: Top gate features by gain. The classifier accurately learns to identify users whose keyword results yield poor coverage of historical interests.

Rank	Feature	Rel. gain	Source
1	Product set density	1.00	Search results
2	Avg. query token count	0.53	Query
3	Result leaf categories	0.30	Search results
4	Semantic L1 coverage	0.27	Coverage gap
5	Brands per category	0.24	Brand-category
6	Result diversity	0.20	Coverage gaps
7	Avg. purchase value	0.16	Purchase value

The LambdaMART reranker orders candidates utilizing 55 unique user, product, and cross-backend matching features. It targets a graded supervision label ranging from 0 to 15 (distinct from the LLM-judge Relevance Score), calculated by combining: (i) semantic prefix-match length across 5 tiers of the same hierarchical SigLIP2 embedding used to derive the gate’s Semantic L1/L2 features (Table 4), and (ii) down-funnel engagement interaction weights (purchase = 3, add-to-cart = 2, view = 1). This dense supervision methodology yields 50× more positive training signals than strict item-ID matching while natively isolating high-intent items.

5 Experimental Evaluation and Ablations

We evaluate SMART end-to-end, validating the routing gate accuracy, system stability, qualitative behaviors, and production live-traffic performance.

5.1 Routing Gate Ablation Studies

Feature Importance Calculations. We train a gradient-boosted gate model to analyze feature contributions. Table 5 tracks relative information gain across top-performing features.

The empirical distribution confirms that search-result features (Product set density, gain = 1.00) and explicit coverage gap metrics (Semantic L1 coverage, gain = 0.27) represent the primary predictive signals, justifying our output-auditing routing design over static entry-history routing.

Operational Quality-Cost Threshold Sweeps. We sweep the gate threshold θ from 0 (always-SMART) to ∞ (keyword-only), mapping performance trajectories across varying LLM routing fractions in Figure 2.

CatRecall@200 demonstrates an inverse-U curve, peaking at 56.6% within the 10% routing band before decaying as further uncalibrated LLM candidates introduce pool dilution and noise. Concurrently, Relevance Score exhibits its steepest trajectory gains within the same 5–15% band, encountering clear diminishing returns thereafter. Setting the gate operating threshold to route exactly 10% of incoming traffic captures the peak CatRecall@200 and the bulk of semantic prospecting gains, lifting Relevance Score from 1.95 (keyword-only) to 2.36—approaching the 2.39 of the standalone LLM + ANN configuration (Table 2); the recall-cost operating points are summarized in Table 6.

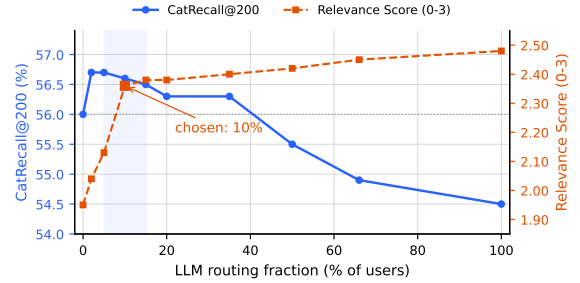


Figure 2: Quality-cost trade-off across LLM routing fraction. CatRecall@200 (blue) peaks within the 5–15% band, while Relevance Score (orange dashed) demonstrates steep, immediate gains within that identical operating range.

Table 6: Operating points at production scale. Gated 10% achieves optimal category recall at a strict 90% LLM infrastructure cost reduction versus always-on routing. Deltas are reported relative to the keyword baseline.

Strategy	LLM%	CatRecall@200	Δ rel.	Rel. cost
Keyword-only (baseline)	0%	56.0%	—	0×
Gated 10%	10%	56.6%	+1.1%	0.1×
Gated 15%	15%	56.5%	+0.9%	0.15×
Always SMART	100%	54.5%	−2.7%	1×
LLM retarget+prospect	100%	42.5%	−24.1%	1×

5.2 Qualitative Case Studies

Table 7 details end-to-end walkthroughs across three distinct consumer profiles, demonstrating how the dual-backend layout resolves systemic failure modes of single-backend configurations.

The pattern is highly consistent: LLM queries append clear semantic precision (User C’s K-beauty surfacing), yet their focus on matching a coherent persona causes them to drop loose, unaligned affinities like User B’s marketplace book purchases. Conversely, keywords + BM25 emits search queries for all historical interactions at zero cost, preserving coverage for tail or high-volume affinities even when displaced from the top slots. SMART’s structural pairing—utilizing the keyword rules as the history-faithful candidate backbone (170 of 200 slots) and the LLM dense path to populate the top ranking layer (30 slots)—mitigates this trade-off in the illustrated cases.

5.3 System Generalization and Stability

Run-to-Run Stability Verification. To ensure the gate captures stable intent rather than artifacts of LLM stochasticity, we execute parallel generation cycles on identical user logs. Table 8 tracks cross-run alignment.

The consistency is strong: 80.1% of users improved in Run 1 are stably improved in Run 2, confirming that user-level benefit from generative expansion is an invariant, learnable property of the request state.

Table 7: Three end-to-end pipeline walkthroughs (advertiser brands anonymized by category role). Each row illustrates a distinct failure mode of single-backend search; SMART resolves all three structurally.

User profile	KW queries → BM25 top-3	LLM queries → ANN top-3
User A: mostly fashion + jewelry; tail browsing (baseball gear, phone jammers, 2–5 events each)	<i>Queries:</i> brand + category for her three main brands, plus tail-noise queries (“baseball helmets”, “phone jammers”). <i>Top-3:</i> three sporting-goods batting helmets/gloves. <i>Failure:</i> tail query monopolizes BM25; her main apparel and jewelry brands fall below rank 10.	<i>Queries:</i> “women’s dress pants” and “silver rings” targeted at her main apparel and jewelry brands, plus persona-grounded paraphrases (“minimalist makeup brush set”). <i>Top-3:</i> three of her main-brand dress pants in different styles. <i>Outcome:</i> main brand recovered; persona-inferred beauty added.
User B: dominant signal marketplace books; secondary signal two Indian ethnic-wear brands	<i>Queries:</i> each brand name verbatim, plus “Print Books”. <i>Top-3:</i> three marketplace book listings (children’s, philosophy, language-learning). <i>Outcome:</i> her dominant books signal is captured at the top of the ranking.	<i>Queries:</i> brand + ethnic-wear vocabulary (“kurta”, “palazzo pants”), plus discovery paraphrases (“ethnic wear kurtis”). <i>Top-3:</i> wide-leg printed palazzo pants from three boho-fashion brands. <i>Outcome:</i> brand-diverse ethnic-wear surface; books discarded.
User C: main brands one fast-fashion + one activewear; “Beauty Maven” persona (no K-beauty in clicks); tail baby-clothing events	<i>Queries:</i> brand + category for her two main brands, plus “Baby & Toddler Clothing”. <i>Top-3:</i> three toddler sleep bags from a single baby-clothing advertiser. <i>Failure:</i> tail baby query dominates BM25; her main brands fall below rank 10.	<i>Queries:</i> activewear paraphrases for her main brand, plus persona-grounded discovery (“serum for dark spots”, “SPF 50 sunscreen”, “hair mask for dry hair”). <i>Top-3:</i> three of her main-brand activewear tops. <i>Outcome:</i> main brand recovered; three new K-beauty brands surfaced via persona expansion.

Table 8: Run-to-run stability of LLM impact. 80.1% of users showing positive metric gains in the initial run maintain positive trajectories in the validation run, confirming the signal is robust.

Run 1 outcome	Still same	Flipped	Neutral
Improved	80.1%	5.1%	14.8%
Declined on CatRecall@200	75.6%	2.2%	22.2%

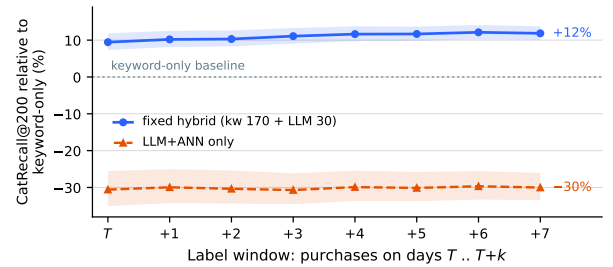
Table 9: Temporal generalization: gate models trained on day T , evaluated on held-out day $T + 1$ user sets.

Model	AUC	CatRecall@5 Δ
GBDT	0.753	+0.99pp
DNN	0.748	+1.13pp
Oracle (per-user)	—	+6.9pp

Temporal Evaluation Cross-Validation. We evaluate gate generalization capacity under a strict temporal protocol: models are trained on day T and tested on completely non-overlapping, held-out day $T + 1$ user sets across distinct catalog states.

Table 9 shows that the gate generalizes across time. GBDT achieves an AUC of 0.753 on unseen-day users, driving a +0.99pp CatRecall@5 lift over baseline. A deep neural network (DNN) variant yields comparable metrics (AUC = 0.748, +1.13pp lift), demonstrating that the coverage-gap predictive signal is robust and model-agnostic.

Robustness to Delayed Purchase Labels. A relevant category may convert only after a delay—over half of the categories a user buys in the week after day T are absent from the day- T label set—so we check whether our single-day CatRecall labels bias the comparison. Holding each user’s history, catalog snapshot, and retrieved top-200 candidates fixed, we expand only the evaluated labels from day T to day T plus the following seven days, and re-score each

**Figure 3: Sensitivity of CatRecall@200 to the purchase-label horizon. Per user, history, catalog snapshot, and retrieved top-200 candidates are fixed at day T ; only the cumulative purchase-category label window expands from T to $T..T+k$. Curves show lift over keyword-only ($= 0$) with pointwise 95% user-bootstrap intervals, on a converter-conditioned cohort. The ordering is stable across all horizons.**

configuration—all retrieving 200 candidates—relative to keyword-only (Figure 3). The day- T ordering is unchanged across horizons: the fixed keyword + LLM hybrid (170 + 30 candidates) holds a stable +9–12% edge over keyword-only (pointwise 95% bootstrap intervals exclude zero at every horizon), while LLM+ANN alone stays well below it (consistent with Section 3.3). This converter-conditioned cohort uses the production catalog and candidate budget, so its recall is not directly comparable to Tables 2 and 6; we read it as a sensitivity check for delayed labels rather than a prospecting measure—longer windows reduce label censoring but admit post- T intent drift, and never-purchased relevant items stay unlabeled, leaving Relevance Score the complementary prospecting metric.

5.4 Online Production Deployment

We validate the framework through a 2-week online A/B test on live Snap traffic against an optimized production baseline consisting of a two-tower user-to-product (U2I) ANN retrieval model evaluating hundreds of features. Production traffic was split uniformly at random between arms. This test also supplies the comparison against a

Table 10: Online A/B production test results (2 weeks). SMART is evaluated directly against Snap’s production embedding-based U2I framework.

Method	CR	CTR
Embedding-based U2I (baseline)	100%	100%
SMART (ours)	127.6%	99.1%

state-of-the-art retriever that our offline studies intentionally omit: Sections 3 and 5.1–5.3 hold indices and rerankers fixed to isolate query-generation effects, whereas here SMART is measured head-to-head against the production embedding-based model under live traffic.

Table 10 logs the live results. SMART delivers a substantial +27.6% relative improvement in core Conversion Rate (CR), defined as attributed click-and-purchase actions, while Click-Through Rate (CTR) remains flat at −0.9%. This performance divergence confirms that SMART is not generating low-intent clicks indiscriminately, but rather surfaces highly relevant discovery items that convert while actively filtering out low-intent tail noise.

6 Discussion, Limitations, and Conclusion

6.1 Discussion

Domain Structural Mismatches. Prior literature on LLM query expansion [15, 18, 27] focuses almost exclusively on long-form, unstructured documents within academic benchmarks like MS-MARCO. Our findings prove that within commercial catalogs characterized by concise, highly structured titles (5–15 tokens), the lexical mismatch introduced by fluid natural-language LLM paraphrasing is significantly more severe. Generative queries do not function as drop-in replacements for traditional search heuristics; instead, they shift candidate retrieval parameters away from strict historical retargeting toward intent-driven semantic discovery. SMART explicitly accounts for this by matching each query style to its winning index engine.

Capability Selection via Output Gating. Prevailing cost-aware routing networks (e.g., FrugalGPT [8], RouteLLM [21]) concentrate on *model selection*, analyzing raw input text difficulty to split queries between cheaper and larger LLM variants. SMART instead operates on *capability selection*. By executing a zero-cost heuristic pass first and auditing its structural output for active coverage gaps, the gate conditionally decides whether to invoke generative machinery at all. This execution pattern generalizes to any high-throughput retrieval workflow combining fast heuristics with expensive neural augmentations.

Generalizability. The transferable core of SMART is the process: decomposing retrieval into retargeting- and prospecting-style objectives, a diagnostic that quantifies who benefits from expensive augmentation, and a gate that audits cheap results before invoking it. The concrete instantiation, however, is catalog-bound: rule-generated queries assume structured brand and category fields, and many gate features derive from category taxonomies or from the semantic-ID hierarchy—though the latter is learned from item embeddings and could be rebuilt in any domain with embeddable

items. Practitioners in domains without such structure would need to substitute their own coverage measures over whatever cheap first-pass retrieval exists, while the audit-then-invoke principle of the preceding paragraph carries over unchanged.

6.2 Limitations and Future Horizons

Several research vectors remain open. First, our temporal validation is currently bounded by next-day holdout testing; extending this tracking across multi-week production horizons will verify systemic robustness against long-term catalog churn and seasonal shopping shifts. Relatedly, the delayed-label robustness check of Section 5.3 extends labels by only one week; longer horizons and attribution of online conversions seeded by each retrieval path remain future work. Second, our offline Relevance Score evaluations utilize a single judge family (Gemini 3 Flash). While we anchor its outputs against real-world conversion metrics to mitigate bias, future configurations will introduce multi-model consensus parsing and human validation panels to eliminate potential model sycophancy. Third, the routing threshold is not load-aware: traffic surges raise LLM call volume proportionally, though service degrades gracefully since every request already completes the keyword pass. Finally, transitioning our binary routing classifier into a continuous regression formulation that directly targets the expected metric delta $\Delta(u)$ will allow for granular, real-time threshold calibration to balance infrastructure cost against fluctuating ad request volume.

6.3 Conclusion

The development and deployment of SMART demonstrates that rule-generated keywords and generative language queries operate as structural complements rather than substitutes in production recommender systems. Deterministic keyword rules exploit inverted index token matching to serve as an optimal candidate foundation for retargeting on BM25. Concurrently, generative language models interpret multi-event intent sequences to perform semantic prospecting over dense ANN vector spaces. By bifurcating retrieval paths through an output-aware quality gate, SMART captures the broad discovery capabilities of LLMs, improving online ad conversions by +27.6% at a strict 90% reduction in total generative infrastructure cost.

References

- [1] Kenya Abe, Kunihiro Takeoka, Makoto P Kato, and Masafumi Oyama. 2025. LLM-based Query Expansion Fails for Unfamiliar and Ambiguous Queries. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3035–3039.
- [2] Eliran Abutbul, Yohay Kaplan, Naama Krasne, Oren Somekh, Or David, Omer Duvdevany, and Evgeny Segal. 2023. Audience Prospecting for Dynamic-Product-Ads in Native Advertising. In *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 1571–1580.
- [3] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, et al. 2024. AutoMix: Automatically Mixing Language Models. *Advances in Neural Information Processing Systems* 37 (2024), 131000–131034.
- [4] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International conference on learning representations*, Vol. 2024. 9112–9141.
- [5] Luiz Bonifácio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. 2022. InPars: Data Augmentation for Information Retrieval using Large Language Models. *arXiv preprint arXiv:2202.05144* (2022).
- [6] Christopher JC Burges. 2010. From RankNet to LambdaRank to LambdaMART: An Overview. *Learning* 11, 23–581 (2010), 81.

- [7] Jaime Carbonell and Jade Goldstein. 1998. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference*. 335–336.
- [8] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).
- [9] Mengxiang Chen, Zhouwei Zhai, and Jin Li. 2026. Probe-then-Plan: Environment-Aware Planning for Industrial E-commerce Search. *arXiv preprint arXiv:2603.15262* (2026).
- [10] Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759.
- [11] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [12] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks Lakshmanan, and Ahmed H Awadallah. 2024. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations*, Vol. 2024. 41348–41366.
- [13] Xin Dong, Sen Jia, Ming Rui Wang, Yan Li, Zhenheng Yang, Bingfeng Deng, and Hongyu Xiong. 2025. COEF-VQ: Cost-Efficient Video Quality Understanding through a Cascaded Multimodal LLM Framework. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4387–4395.
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. The Faiss Library. *IEEE Transactions on Big Data* (2025).
- [15] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1762–1777.
- [16] Gemini Team, Google. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805* (2023).
- [17] Nan Hu, Han Li, Jimeng Sun, Lu Wang, Fangkai Yang, Bo Qiao, Pu Zhao, David Dai, Mengyu Liu, Yuefeng Zhan, et al. 2026. AdNanny: One Reasoning LLM for All Offline Ads Recommendation Tasks. *arXiv preprint arXiv:2602.01563* (2026).
- [18] Rolf Jagerman, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bender-sky. 2023. Query expansion by prompting large language models. *arXiv preprint arXiv:2305.03653* (2023).
- [19] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in neural information processing systems* 30 (2017).
- [20] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. 2021. Sparse, Dense, and Attentional Representations for Text Retrieval. *Transactions of the Association for Computational Linguistics* 9 (2021), 329–345.
- [21] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024).
- [22] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, Maciej Kula, Ed H Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [23] Stephen Robertson and Hugo Zaragoza. 2009. *The Probabilistic Relevance Framework: BM25 and Beyond*. Vol. 4. Now Publishers Inc.
- [24] Tantivy Contributors. 2023. Tantivy: A Full-Text Search Engine Library Written in Rust. <https://github.com/quickwit-oss/tantivy>.
- [25] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. 2025. SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features. *arXiv preprint arXiv:2502.14786* (2025).
- [26] Han Wang, Mukuntha Narayanan Sundararaman, Onur Gungor, Yu Xu, Krishna Kamath, Rakesh Chalasani, Kurchi Subhra Hazra, and Jinfeng Rao. 2024. Improving Pinterest Search Relevance Using Large Language Models. *arXiv preprint arXiv:2410.17152* (2024).
- [27] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9414–9423.
- [28] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [29] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2025. Large Language Models for Information Retrieval: A Survey. *ACM Transactions on Information Systems* 44, 1 (2025), 1–54.